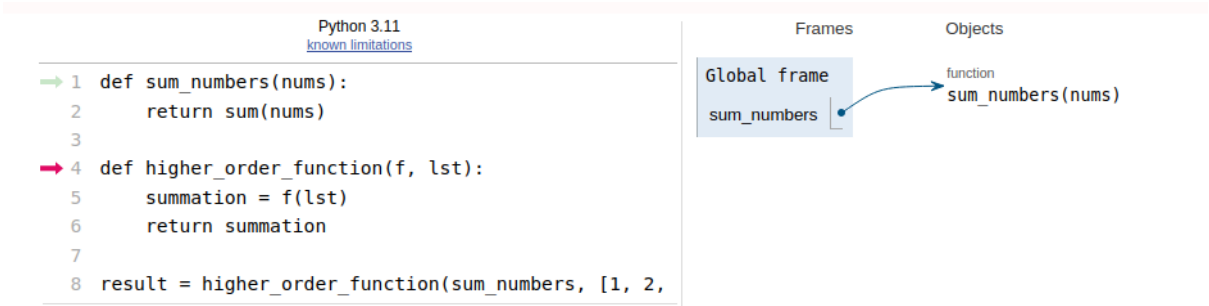
```python
def sum_numbers(nums):
    return sum(nums)

def higher_order_function(f, lst):
    summation = f(lst)
    return summation

result = higher_order_function(sum_numbers, [1, 2, 3, 4, 5])
```

Introducción: El Contexto de Ejecución.



1. Lectura e Identificación de la Declaración

Antes de que se ejecute la primera línea de código, el intérprete de Python (por ejemplo, CPython) crea un entorno de ejecución. Lo primero que hace es establecer el **Espacio de Nombres Global (o *Global Namespace*),** que en la imagen se visualiza como el "Global frame".

Piensa en este espacio de nombres global como un diccionario interno de Python. Al principio, este diccionario contiene algunas entradas por defecto (como **__name__**, **__doc__**, **__package__**, **etc.**), pero no tiene ninguna de las variables de tu código.

2. Creación del Objeto de Función (en el *Heap*)

Python es un lenguaje orientado a objetos. Todo en Python es un objeto, ¡incluso las funciones! Al encontrarse con **def sum_numbers(nums):**, el intérprete realiza las siguientes acciones:

- **Asignación de Memoria:** Reserva un bloque de memoria en el "montón" (o *heap*, que es el panel de "Objects" en la imagen) para un nuevo objeto.
- **Creación del Objeto:** Crea un nuevo **objeto de función** (un objeto de tipo **function**). Este objeto es una estructura de datos compleja que encapsula varias cosas:
    - **Código de Bytes (Bytecode):** Una versión compilada de las líneas 1 y 2 (el cuerpo de la función). Este bytecode se almacena dentro del objeto de función, pero *no se está ejecutando* en este momento.

- **Metadatos de la Función:** Almacena el nombre de la función ("sum_numbers"), la lista de sus parámetros formales (nums), sus valores predeterminados (si los hubiera), sus anotaciones de tipo, etc.
- **Referencia al Ámbito (Scope):** Almacena una referencia al espacio de nombres en el que se definió la función. En este caso, es el Global Namespace. Esto es crucial para que la función pueda acceder a variables globales más tarde.

3. Enlace de Nombre (Name Binding) en el Global Namespace

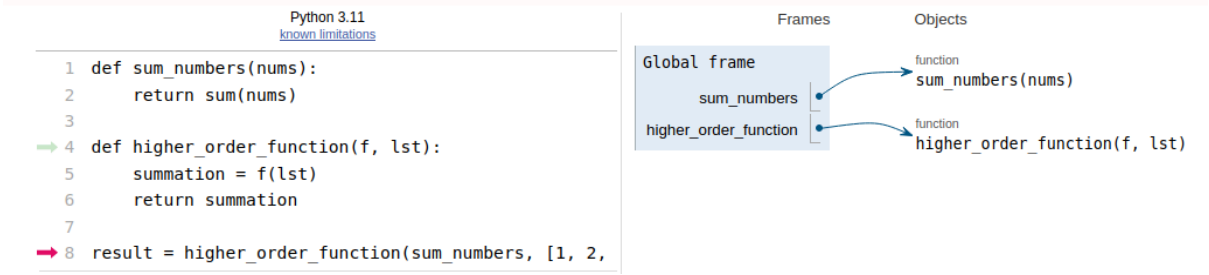
Ahora que el objeto de función existe en la memoria, el intérprete debe darle un nombre para que puedas usarlo. Para ello, actualiza el **diccionario del Espacio de Nombres Global**:

- **Clave:** Inserta una nueva clave en el diccionario: la cadena de texto **'sum_numbers'**.
- **Valor:** Asocia a esa clave una **referencia de objeto** (un puntero) que apunta directamente al objeto de función recién creado en el heap.

Conectando con la Imagen

Ahora, mira la visualización con este conocimiento técnico:

- **La Flecha Verde:** Confirma que todo este proceso de definición ha finalizado para la línea 1.
- **Global frame (Global Namespace):** Es el diccionario interno de Python.
- **sum_numbers (en el Global frame):** Es la entrada del diccionario. El nombre de la función es ahora una variable en el ámbito global.
- **El Puntero Azul:** Visualiza la **referencia de objeto**. El nombre **sum_numbers** en el Global frame no *es* la función; es simplemente un puntero a la función.
- **El Objeto en Objects (en el *Heap*):** Es el objeto de función real que se ha creado en la memoria. Python Tutor lo etiqueta de forma simplificada como **function sum_numbers(nums)**, pero técnicamente es esa estructura de datos compleja que contiene el bytecode, los metadatos y la referencia al ámbito.



Análisis Técnico del Paso 2: Procesando la Línea 4 (def higher_order_function(f, lst):)

1. Lectura e Identificación de la Declaración

El intérprete, tras definir la primera función, busca la siguiente instrucción ejecutable de nivel superior. Salta el cuerpo de `sum_numbers` y llega a la línea 4. Reconoce nuevamente la palabra clave `def`. Al igual que en el Paso 1, esto es una **instrucción de definición de función**, no una invocación. Su propósito es crear una segunda estructura conceptual para usarla más adelante.

2. Creación del Segundo Objeto de Función (en el *Heap*)

Python crea un nuevo objeto en el "montón" de memoria (*heap*, panel de "Objects").

- **Asignación de Memoria:** Reserva un segundo bloque de memoria para un nuevo objeto.
- **Creación del Objeto:** Se crea un nuevo **objeto de función** (un segundo objeto de tipo `function`), totalmente **distinto e independiente** del primero. Esta estructura de datos encapsula:
    - **Código de Bytes (Bytecode):** La versión compilada de las líneas 5 y 6 (el cuerpo de la función). Este bytecode se almacena dentro de este nuevo objeto, pero **no se está ejecutando**.
    - **Metadatos de la Función:** Almacena el nombre de la función ("higher_order_function"), la lista de sus parámetros formales (`f`, `lst`), valores predeterminados, anotaciones de tipo, etc.
    - **Referencia al Ámbito (Scope):** Almacena una referencia al Espacio de Nombres Global, para saber dónde se definió.

3. Enlace de Nombre (Name Binding) en el Global Namespace

Ahora que el segundo objeto de función existe en la memoria, el intérprete debe darle un nombre en el mismo espacio de nombres global (Global Frame). Para ello, actualiza el **diccionario interno del Espacio de Nombres Global**:

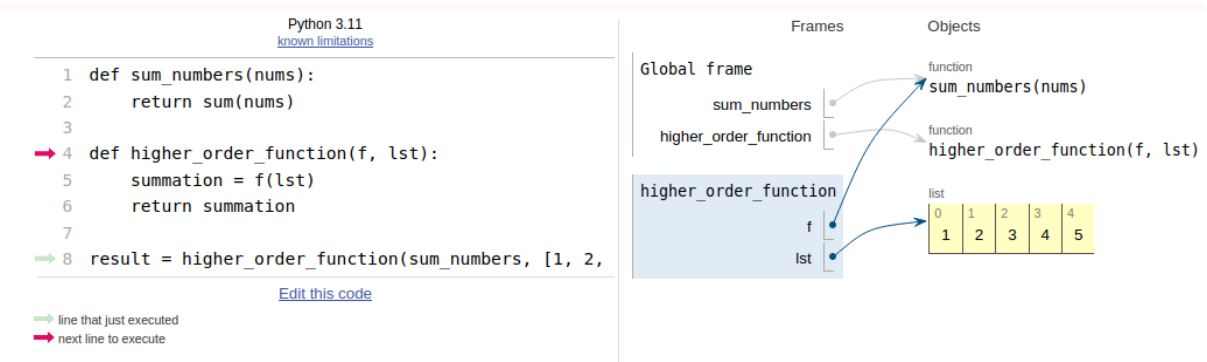
- **Clave:** Inserta una nueva clave en el diccionario: la cadena de texto `'higher_order_function'`.
- **Valor:** Asocia a esa clave una **referencia de objeto** (un segundo puntero) que apunta directamente al nuevo objeto de función recién creado en el heap.

Conectando con la Imagen

Mira la visualización con este conocimiento técnico para el Paso 2:

- **La Flecha Verde:** Confirma que la definición de la línea 4 ha finalizado.
- `Global frame` **(Global Namespace):** El diccionario global ahora contiene dos entradas. Ya no es solo `sum_numbers`, sino que ahora se ha añadido `higher_order_function`.
- **La Entrada** `higher_order_function`**:** Es la nueva variable que representa la referencia.

- **El Segundo Puntero Azul:** Visualiza la **referencia del objeto**. El nombre `higher_order_function` es simplemente un puntero a un objeto de función.
- **El Segundo Objeto en `Objects` (en el *Heap*):** Hay ahora dos objetos de función distintos en la memoria. El nuevo objeto, conceptualmente, es el que Python Tutor etiqueta como `function higher_order_function(f, lst)`.



Análisis Técnico del Paso 3:

Procesando la Línea 8 (`result = higher_order_function(sum_numbers, [1, 2, 3, 4, 5])`)

1. Activación de la Invocación (Inicia la llamada a la función)

El intérprete llega a la línea 8. La instrucción es una asignación (`result = ...`). Para resolverla, primero debe evaluar el lado derecho de la ecuación (`=`): la llamada a la función.

- **Búsqueda de la Función:** Python mira en el **Espacio de Nombres Global (Global frame)** para encontrar a qué se refiere `'higher_order_function'`. Sigue el puntero (la referencia) hacia el **objeto de función** que creamos en el Paso 2 (panel "Objects"). Python ahora sabe qué código debe ejecutar.

2. Evaluación e Instanciación de Argumentos (Preparación de datos)

Antes de que la función pueda correr, Python debe preparar los datos que le está pasando. Esto es un proceso detallado de **evaluación de argumentos**:

- **Argumento 1 (`sum_numbers`):**
  - Python busca `'sum_numbers'` en el Global frame. Sigue el puntero y obtiene una referencia al **objeto de función** `sum_numbers`.
  - **Tecnicismo Importante:** No estamos *ejecutando* `sum_numbers`; estamos pasándole la *función en sí misma* como si fuera un dato. Este es un principio de las funciones de orden superior.
- **Argumento 2 (`[1, 2, 3, 4, 5]`):**

- **Instanciación del Objeto Lista:** Python ve un literal de lista. Reserva un nuevo bloque de memoria en el "montón" de memoria (heap/Objects panel) para un nuevo **objeto de tipo `list`**.
- **Poblado del Objeto:** Este objeto lista se crea como una estructura de datos (un array de punteros) que contiene referencias a los objetos enteros 1, 2, 3, 4, 5.
- **Referencia Resultante:** Python ahora tiene una referencia a este nuevo objeto lista. Esta referencia es lo que se pasará a la función. En la imagen, este objeto lista aparece en el panel "Objects" con sus índices y valores.
- **[Ver Apunte de arreglos]: https://docs.google.com/document/d/1giWPrsosNPggWeUThTHo5onpmZyvAtlP5GDv7W89Hck/edit?tab=t.0**

3. Creación del Nuevo Frame (Entorno Local)

Una vez preparados los argumentos, Python "empuja" un nuevo **entorno de ejecución (o frame)** para **`higher_order_function`** en la **pila de llamadas (call stack)**.

- **Asignación de Memoria:** Se reserva memoria para un nuevo diccionario de **Espacio de Nombres Local**. En la imagen, esto se visualiza como un segundo rectángulo dentro de 'Frames', titulado `higher_order_function`.
- **Cambio de Ámbito:** Python ahora opera dentro de este nuevo frame local. Ya no mira directamente al frame global, sino a este frame local *primero* para cualquier variable que no esté precedida por `global`.

4. Vinculación de Parámetros (Parameter Binding)

Este es el paso crucial donde los datos que preparamos se "asignan" a los nombres de las variables de la función. Esto no es una copia de datos, sino una **vinculación de referencias** (pointing):

- **Clave Local `'f'`:** En el nuevo frame local, Python inserta una clave `'f'`. A esta clave, le asigna la **referencia (puntero)** al objeto de función `sum_numbers` que preparamos en el paso 2. El puntero azul en la imagen muestra esta vinculación: `f` local apunta al mismo objeto que `sum_numbers` global.
- **Clave Local `'lst'`:** En el nuevo frame local, Python inserta una clave `'lst'`. A esta clave, le asigna la **referencia (puntero)** al objeto lista recién creado (`[1, ..., 5]`). El otro puntero azul en la imagen muestra esta vinculación: **`lst`** local apunta al objeto lista en el heap.

5. Actualización del Flujo de Control

Como el nuevo entorno (frame) está configurado e inicializado correctamente, Python está listo para ejecutar el cuerpo de la función.

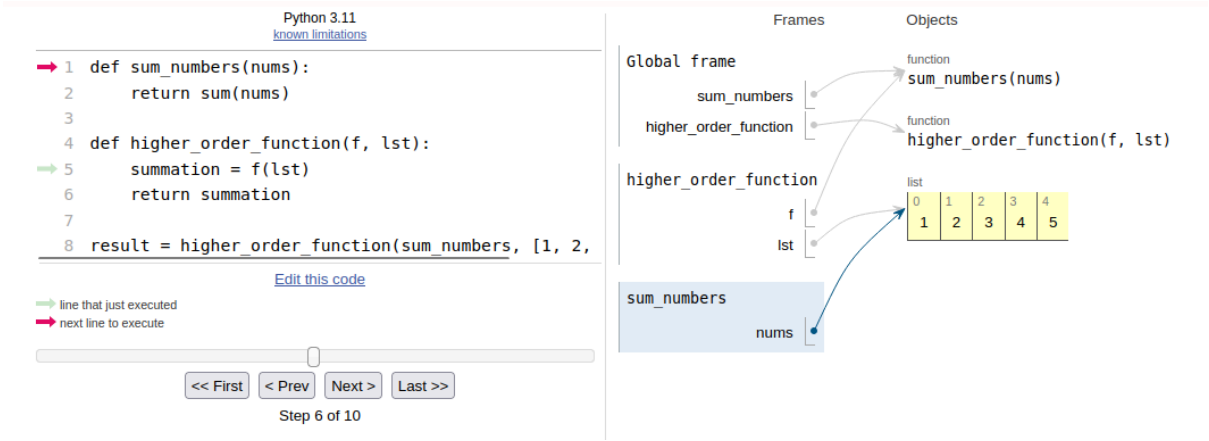
- **Mover el Puntero de Ejecución:** El flujo de ejecución ha saltado desde la línea 8 (el llamador) hacia *adentro* de la función (el llamado). La flecha roja de "next line to execute" se mueve a la primera línea ejecutable del cuerpo de la función: la línea 5

(`summation = f(lst)`). En la imagen, la flecha roja aparece en la línea 4 (el encabezado) porque Python Tutor marca el encabezado al *entrar* al frame, justo antes de ejecutar la primera línea de código del cuerpo, para indicar el cambio de contexto.

Conectando con la Imagen

Mira la visualización final con este conocimiento técnico:

- **`Global frame` (Global Namespace):** El diccionario global sigue intacto.
- **`higher_order_function` (en Frames):** Este es el nuevo **entorno de ejecución local**. Es un espacio de memoria aislado y temporal para la función que se está ejecutando actualmente.
- **Las Variables Locales (`f`, `lst`):** Son las entradas en el diccionario local.
- **Los Dos Punteros Azules:** Son **vinculaciones de referencia**. Demuestran cómo la función de orden superior ha recibido acceso al objeto de función (`f` apunta a `sum_numbers`) y al objeto lista (`lst` apunta a la lista en el heap).
- **El Objeto Lista en `Objects` (en el *Heap*):** Es el objeto físico en la memoria que se creó para contener `[1, ..., 5]`.
- **La Flecha Verde:** Muestra que la línea 8 (la llamada) se ha activado y completado para dar paso a la entrada en la función.
- **La Flecha Roja:** Confirma que el intérprete está ahora *dentro* de `higher_order_function`, listo para ejecutar la línea 5 en su siguiente paso.



Análisis Técnico Profundo del Paso 6.

1. El Contexto de Ejecución (Pila de Llamadas)

Mira la columna "Frames". En este preciso instante, la **pila de llamadas (call stack)** contiene tres entornos (marcos) distintos:

1. **`Global frame`:** El entorno base de todo el programa. Contiene las definiciones globales (las referencias a los objetos de función de `sum_numbers` y `higher_order_function`) y eventualmente contendrá `result`.
2. **`higher_order_function`:** Este marco local se creó en la llamada anterior. Ya ha ejecutado la línea 5 (`summation = f(lst)`), que es el detonante de todo lo que sigue. Como puedes ver, tiene sus propias variables locales (`f` y `lst`) que apuntan a sus respectivos objetos.
3. **`sum_numbers` (El Nuevo Marco):** Este es el entorno **recién creado** en la pila. Se creó en el Paso 5, inmediatamente después de que el intérprete viera la instrucción de llamada `f(...)` en la línea 5. Es un espacio de memoria aislado para la función `sum_numbers`.

2. La Vinculación de Parámetros (Parameter Binding)

Este es el proceso más detallado y el más crucial del Paso 6. Cuando se creó el marco local para `sum_numbers`, Python tuvo que poblarlo con sus parámetros.

1. **Resolución de la Referencia de Función:** El intérprete leyó la instrucción de llamada `f(...)` dentro de `higher_order_function`. Buscó la variable local `f` y encontró que es un puntero (referencia) al objeto de función `sum_numbers` que reside en el heap.
2. **Evaluación de Argumentos:** Python leyó el argumento `(lst)`. Buscó la variable local `lst` dentro de `higher_order_function` y obtuvo una referencia al objeto lista (`[1, 2, 3, 4, 5]`) que está en el heap.
3. **Inyección en el Nuevo Marco:**
   - Python insertó la clave `'nums'` (el nombre del parámetro formal de `sum_numbers`) en el diccionario del nuevo espacio de nombres local.
   - A esta clave, le asignó el valor de la referencia obtenida en el paso anterior.
   - **Puntero de Referencia:** El **puntero azul** que va desde `nums` local hasta el objeto lista es la visualización de este proceso. **Técnicamente: `sum_numbers.nums` es ahora un alias lógico para el mismo objeto de memoria que `higher_order_function.lst`.** No hay copia de datos; solo una copia del puntero de memoria.

3. Actualización del Flujo de Control (Punteros de Instrucción)

Las flechas en el código visualizan el estado de los punteros de instrucción (instruction pointers) en la pila de llamadas.

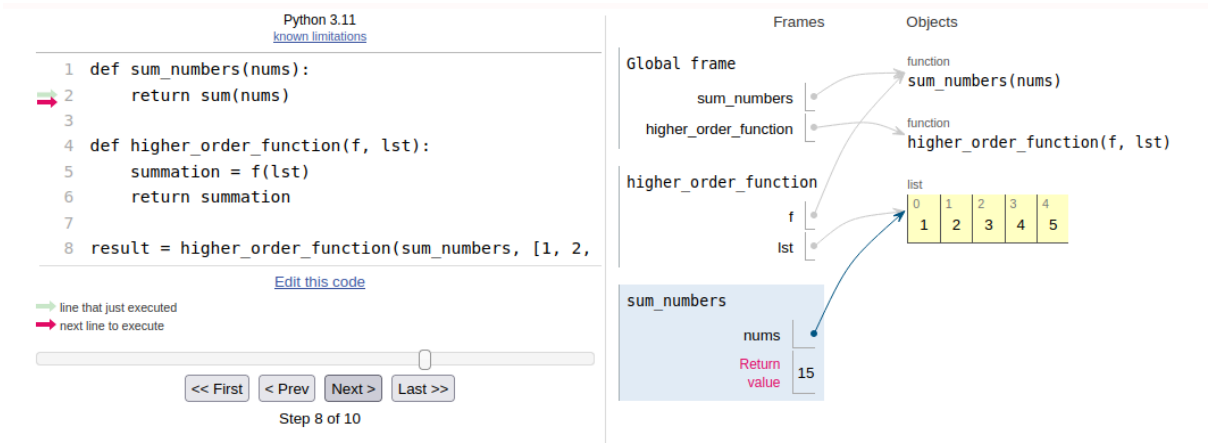
- **Flecha Verde (Línea 5 - `summation = f(lst)`):** Muestra el estado del llamador (`higher_order_function`). El intérprete en este marco ha terminado su última tarea (evaluar argumentos y crear el nuevo marco). El frame de `higher_order_function` está **en espera (bloqueado)**, esperando que el frame que está encima termine y devuelva un valor para poder asignar la variable local `summation`.

- **Flecha Roja (Línea 2 - `return sum(nums)`):** Muestra el estado del llamado (`sum_numbers`). Es la **siguiente instrucción a ejecutar**. El intérprete acaba de entrar a este frame y ha movido el foco de ejecución a la primera línea del cuerpo de la función. El código `return sum(nums)` se ha cargado en el registro de instrucciones, pero el intérprete aún no ha iniciado su evaluación real.

Conectando con la Imagen

Mira la visualización final con este conocimiento técnico:

- **Marcos Locales:** Tienes tres rectángulos que representan tres espacios de nombres independientes en la pila. El más nuevo y "activo" está arriba.
- **Punteros Azules Locales:** Demuestran que `f` y `lst` en un marco apuntan a objetos complejos en el heap, y que `nums` en el marco superior ha sido configurado para apuntar al mismo objeto lista.
- **Heap de Objetos:** Tienes dos objetos de función distintos (que contienen bytecode y metadatos) y un objeto lista (que contiene un array de punteros a objetos enteros) en la memoria compartida.
- **Flechas de Ejecución:** Visualizan la jerarquía de llamadas. La flecha verde muestra quién llamó y está esperando; la flecha roja muestra quién ha sido llamado y está a punto de empezar.



Análisis Técnico Profundo del Paso 8

La flecha verde clara en la línea 2 y la flecha roja en la línea 5 son las claves principales para entender este momento.

1. El Contexto de Ejecución (La Pila de Llamadas)

Mira la columna "Frames". En este preciso instante, la **pila de llamadas (call stack)** todavía contiene tres marcos distintos, pero el superior está a punto de ser eliminado.

1. **`Global frame`:** El entorno base que eventualmente contendrá `result`. Está en espera.
2. **`higher_order_function` (El Marco Bloqueado):** Este marco se creó en la llamada inicial. Ejecutó la línea 5 (`summation = f(lst)`), que es el detonante de todo lo que sigue. Como puedes ver en el código, la flecha verde clara está en la línea 5, lo que indica que esta función hizo la llamada y está **bloqueada (waiting)**, esperando que la función que está encima termine y devuelva un valor para poder asignar ese valor a la variable local `summation`.
3. **`sum_numbers` (El Marco que Retorna):** Este es el entorno local para la función que acaba de terminar su ejecución. Contiene su parámetro `nums`, que apunta al objeto lista. La clave aquí es la nueva entrada **`Return value`** con el valor **15**. Esto significa que el intérprete ha terminado de ejecutar el cuerpo de la función.

2. La Ejecución y el Cálculo del Valor de Retorno

¿Qué sucedió técnicamente en el Paso 7 para llegar a este Paso 8?

1. El intérprete ejecutó la instrucción en la línea 2: `return sum(nums)`.
2. **Llamada a una Función Incorporada (`sum`):** Para evaluar esto, el intérprete Python Tutor hace una llamada interna a la función incorporada `sum()`.
    - Le pasó a `sum()` la referencia que tiene `sum_numbers.nums` (la dirección de memoria de la lista `[1, ..., 5]`).
3. **Cálculo de la Suma:** La función incorporada `sum()` iteró sobre la lista, acumulando los valores enteros (1 + 2 + 3 + 4 + 5), lo que dio como resultado un nuevo objeto entero en el heap con el valor **15**.
4. **Vinculación del Valor de Retorno:** Una vez que `sum(nums)` se evaluó a 15, el intérprete asoció este valor con el registro de "valor de retorno" interno del marco `sum_numbers`. Esto es lo que ves visualizado como `Return value`.

3. El Momento de la Imagen: Retorno del Control

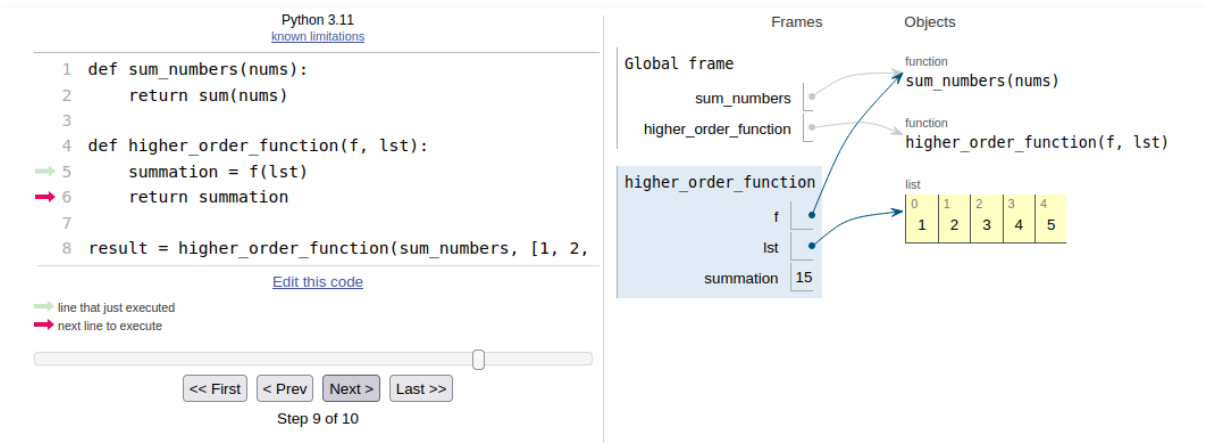
El estado que ves es justo **después** de calcular el valor de retorno, pero **antes** de que el marco se elimine por completo de la pila.

- **Finalización de la Instrucción `return`:** El intérprete ha ejecutado la última instrucción del cuerpo. Por eso la flecha roja (next line) ya no está dentro del cuerpo.
- **Transferencia de Control y Valor:**
    - En el siguiente micro-paso (Paso 9), el marco `sum_numbers` se eliminará por completo de la memoria (pop de la pila).
    - **El valor de retorno (15) se transferirá** desde el registro de retorno de `sum_numbers` directamente a la variable local `summation` que está esperando en el marco de `higher_order_function`.
- **Actualización del Flujo:** El control de ejecución (el instruction pointer del intérprete) volverá a `higher_order_function`. La flecha roja de "next line to execute" se moverá a la línea 6 (`return summation`), ya que la línea 5 se habrá completado.

Conectando con la Imagen

Mira la visualización final con este conocimiento técnico:

- **Pila de Marcos:** Tienes tres rectángulos que representan tres espacios de nombres independientes en la pila. El más nuevo está en proceso de salir.
- **Registro de Valor de Retorno:** Es la entrada clave `Return value: 15`. Es el "fruto" del trabajo de esta función.
- **Punteros Azules Locales:** Demuestran que `sum_numbers.nums` y `higher_order_function.lst` siguen apuntando al mismo objeto lista.
- **Flechas de Ejecución:** Visualizan la jerarquía de llamadas. La flecha verde muestra quién llamó y está esperando; la flecha roja muestra quién ha sido llamado, ha terminado y está en el proceso de entregar el control de vuelta.



Análisis Técnico Profundo del Paso 9

La flecha verde clara en la línea 5 y la flecha roja en la línea 6 son las claves principales para entender este momento.

1. El Contexto de Ejecución (Pila de Llamadas Reanudada)

Mira la columna "Frames". En este preciso instante, la pila de llamadas se ha simplificado.

1. **Eliminación del Marco:** El marco local `sum_numbers` que estaba arriba en el Paso 8 ha sido **eliminado (popped)** de la memoria de la pila. Python ha liberado ese espacio de memoria.
2. **`Global frame`:** Sigue esperando, intacto.
3. **`higher_order_function` (El Marco Reanudado):** Este es el marco local **actualmente activo** en la cima de la pila. Como puedes ver en el código, la flecha verde clara está en la línea 5 (`summation = f(lst)`), lo que indica que esta instrucción **se ha completado por completo**. La función ya no está bloqueada.

2. La Asignación del Valor de Retorno

¿Qué sucedió técnicamente en el Paso 8 y en la transición a este Paso 9 para llegar aquí?

1. **Transferencia de Valor:** El intérprete de Python tomó el valor de retorno final que calculamos en el paso anterior (el objeto entero **15**).
2. **Vinculación Local:** Inmediatamente después de eliminar el marco `sum_numbers`, el intérprete insertó la clave `'summation'` en el diccionario del espacio de nombres local de `higher_order_function`.
3. **Poblado de Datos:** Asignó el valor entero 15 a esta nueva clave.
    - **Resultado:** En la imagen, el marco `higher_order_function` ahora tiene tres entradas locales: `f` (puntero a función), `lst` (puntero a lista) y **`summation: 15`**. Esta es la visualización de que la asignación local se ha realizado con éxito.

3. Actualización del Flujo de Control

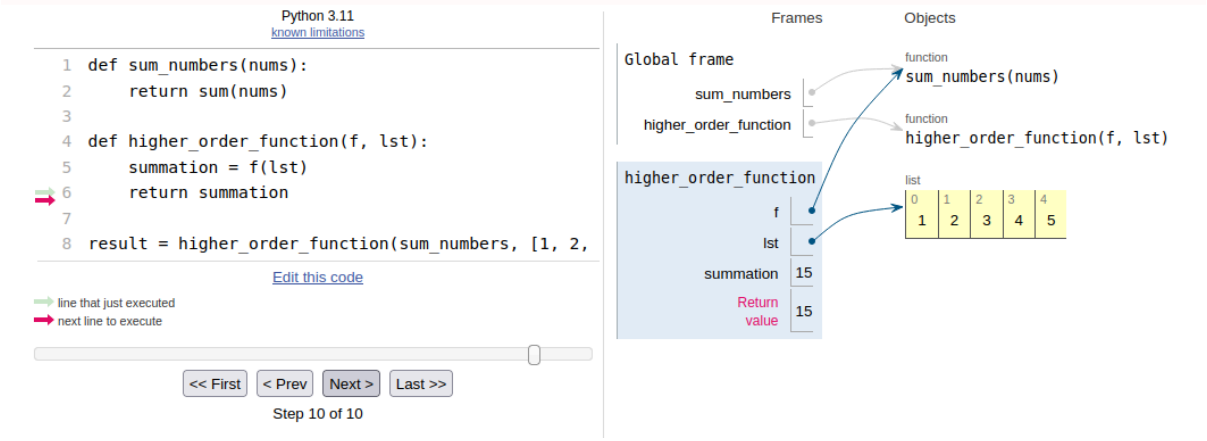
Como la instrucción en la línea 5 se ha completado (evaluación + asignación), el intérprete debe avanzar.

- **Siguiente Instrucción:** El control de ejecución (el puntero de instrucción del intérprete) se mueve a la siguiente línea del cuerpo de la función. La flecha roja de "next line to execute" se mueve a la línea 6: **`return summation`**.
- **Estado de Espera:** El intérprete ha cargado esta instrucción en el registro de instrucciones y está listo para ejecutarla, pero aún no lo ha hecho.

Conectando con la Imagen

Mira la visualización final con este conocimiento técnico:

- **Pila de Marcos:** Tienes dos rectángulos. El que estaba arriba ha desaparecido.
- **Variable Local `summation`:** Esta es la entrada clave `summation: 15`. Demuestra que el marco llamador ha recibido el resultado de la función que llamó y lo ha almacenado localmente.
- **Punteros Azules Locales:** Demuestran que `f` y `lst` en este marco siguen apuntando a sus objetos complejos en el heap, y que `higher_order_function.lst` sigue siendo el único alias para la lista.
- **Flechas de Ejecución:** Visualizan el estado del flujo. La flecha verde muestra qué instrucción se completó para llegar aquí; la flecha roja muestra qué instrucción es la siguiente en la cola.



Análisis Técnico Profundo del Paso 10

La flecha verde clara en la línea 6 y la flecha roja en la línea 8 son las claves principales para entender este momento final.

1. El Contexto de Ejecución (Pila de Llamadas Completa y Terminada)

Mira la columna "Frames". Este es el estado final de la memoria de la pila.

1. **Doble Eliminación de Marcos (Pop de la Pila):** En la transición del Paso 9 al 10, Python ha realizado una acción de limpieza doble.
    - Primero, se eliminó el marco local `higher_order_function` de la cima de la pila, liberando toda su memoria (incluyendo sus variables locales `f`, `lst` y `summation`).
    - **Resultado:** Solo queda un marco en la pila: el **`Global frame`**. La pila de llamadas está técnicamente vacía de marcos locales, lo que indica que no hay ninguna función ejecutándose actualmente.

2. La Asignación Global Final

¿Qué sucedió técnicamente para llegar a este estado final en el Paso 10?

1. **Transferencia de Valor Final:** El intérprete de Python tomó el último valor de retorno que calculamos (el objeto entero **15**) del registro de retorno interno del marco `higher_order_function` justo antes de eliminarlo.
2. **Vinculación Global:** El intérprete insertó la clave `'result'` en el diccionario del espacio de nombres global (Global frame).
3. **Poblado de Datos:** Asignó el valor entero 15 a esta clave global.
    - **Resultado:** En la imagen, el `Global frame` ahora tiene tres entradas: `sum_numbers` (puntero a función), `higher_order_function` (puntero a función) y **`result: 15`**. Esta es la visualización de que la asignación final de la línea 8 se ha realizado con éxito. El resultado del programa está ahora almacenado de forma permanente en el ámbito global.

3. Finalización del Flujo de Control

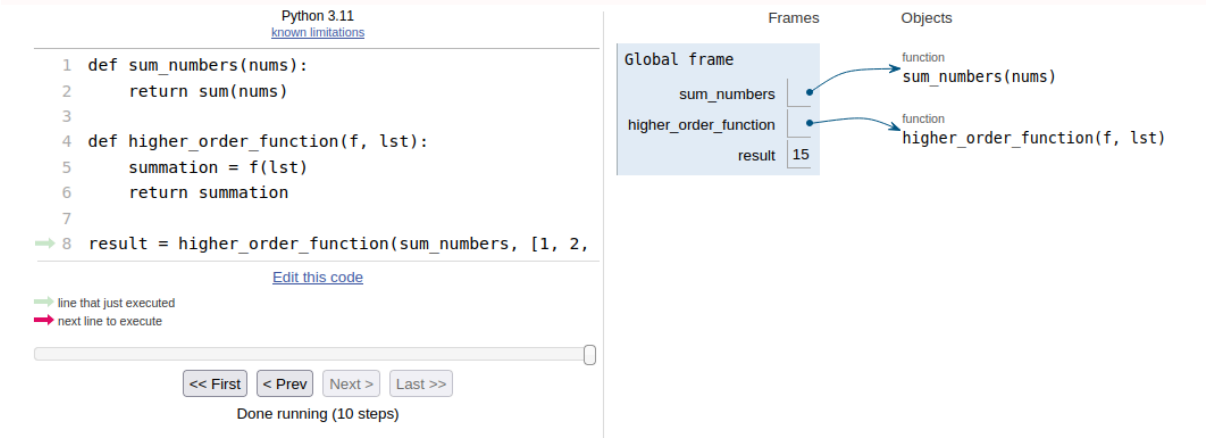
Como la instrucción en la línea 8 (`result = ...`) se ha completado (evaluación + asignación final), el intérprete ha llegado al final de la línea.

- **Sin Más Instrucciones:** La flecha verde clara en la línea 6 (`return summation`) marca el último código de función que se completó.
- **Final del Programa:** La flecha roja de "next line" apunta de nuevo a la línea 8, pero no hay más código que ejecutar después de ella en el ámbito global. Esto indica que el intérprete ha llegado al final del script y está a punto de salir. En un entorno real, esta sería la última instrucción antes de que el proceso Python finalice.

Conectando con la Imagen

Mira la visualización final con este conocimiento técnico:

- **Pila de Marcos:** Solo tienes un rectángulo. Todos los marcos locales temporales han desaparecido.
- **Variable Global `result`:** Esta es la entrada clave `result: 15`. Demuestra que el ámbito global ha recibido el resultado final de toda la cadena de llamadas a funciones y lo ha almacenado.
- **Punteros Azules:** Demuestran que `sum_numbers` y `higher_order_function` en el marco global siguen apuntando a sus objetos de función en el heap (estos objetos se mantendrán hasta que el programa termine), pero no hay más punteros a la lista `[1, ..., 5]`.
- **Recolección de Basura:** Un detalle técnico sutil: como ya no hay ninguna variable local en ningún marco apuntando a la lista, Python Tutor la ha "recolectado como basura" (GC) y la ha eliminado de la visualización para mostrar que esa memoria ha sido liberada.
- **Flechas de Ejecución:** Visualizan la finalización. La flecha verde muestra la última instrucción de función que se completó; la flecha roja muestra que el flujo ha vuelto a la línea 8 pero no tiene adónde ir.



Análisis Técnico Profundo del Paso 11

Este paso es la culminación de la ejecución, caracterizado por la ausencia de instrucciones pendientes.

1. El Contexto de Ejecución Final (La Pila y el Heap)

Mira la columna "Frames". Este es el estado final de la memoria del programa justo antes de terminar.

1. **Limpieza Total de la Pila:** En la transición del Paso 10 al 11, Python Tutor ha realizado la acción de limpieza final.
    - Como el intérprete llegó al final de la línea 8 y no hay más código de nivel superior, el **`Global frame`** (el espacio de nombres global) se ha eliminado o vaciado por completo de variables locales.
    - **Resultado:** En la imagen, el panel "Frames" está **completamente vacío**. No queda rastro de variables como `result`, `sum_numbers` o `higher_order_function` en la pila. La pila de llamadas está físicamente vacía.

2. La Recolección de Basura Final (Garbage Collection)

¿Qué sucedió técnicamente con los objetos en el heap para llegar a este estado final?

1. **Detección de Referencias:** Al vaciarse el Global frame, las variables globales (`sum_numbers`, `higher_order_function` y `result`) que apuntaban a los objetos en el heap dejaron de existir.
2. **Conteos de Referencias a Cero:**
    - El objeto de función `sum_numbers` (en el heap/Objects panel) vio cómo su contador de referencias bajaba a cero (ya no hay ninguna variable local ni global que apunte a él).
    - Lo mismo sucedió con el objeto de función `higher_order_function`.

    - El objeto entero que contenía el valor `15` (`result`) también vio bajar su conteo de referencias a cero.
3. **Ejecución del Garbage Collector (GC):** El recolector de basura de Python identificó estos objetos con conteo cero y liberó la memoria que ocupaban en el heap.
    - **Resultado:** En la imagen, el panel "Objects" también está **completamente vacío**. No quedan objetos de función, ni listas, ni enteros en el heap de objetos.

3. Finalización y Salida del Proceso

Como no hay más instrucciones y la memoria ha sido liberada:

- **Punto de Salida del Script:** El control de ejecución (el instruction pointer del intérprete) ha salido oficialmente del script principal.
- **Ausencia de Flechas de Ejecución:** Por primera vez en todo el proceso, no hay flechas verdes ni rojas en el código. Esto indica que no hay ninguna línea que acabe de ejecutarse ni ninguna que sea la siguiente en la cola. La ejecución ha finalizado de forma natural.
- **Pausado en la Terminación:** El estado "Done running" que muestra Python Tutor indica que el programa ha terminado y estamos visualizando el estado de la memoria en ese momento de "pausa" final antes de que el proceso Python Tutor cierre la sesión por completo. En un entorno real, esta sería la milésima de segundo antes de que el sistema operativo recupere todo el proceso Python.

Conectando con la Imagen

Mira la visualización final con este conocimiento técnico:

- **Paneles Vacíos:** Los paneles "Frames" y "Objects" están en blanco. Esto simboliza la **devolución de toda la memoria** al sistema operativo. Python Tutor ha vaciado la visualización de la memoria para mostrar un estado de "limpieza final" antes de cerrar.
- **Pausado en el Final:** El indicador de paso muestra "Done running". Esto nos dice que el control de ejecución ha terminado y estamos mirando la "fotografía" de la memoria en ese estado de finalización.